Paper B v0.1

2026-05-12

Steering potentials, not bug fixes, eliminate catastrophic outliers in Boltz-2 cofold protein-ligand affinity prediction

A six-way protocol evaluation on zinc-hydroxamate MMP-1 inhibitors

Han Cheongwoo

Draft v0.1 — 2026-05-12

Abstract

End-to-end protein-ligand structure prediction with affinity heads (Boltz-2, AlphaFold3-class cofolding) has become a routine first-pass tool for virtual screening, but the rate at which these models emit physically implausible poses — and how that rate depends on inference-time configuration — remains incompletely characterised. We evaluate six published cofold configurations on 15 ChEMBL MMP-1 zinc-hydroxamate active-site ligands (1500 cofold poses per condition; 9000 total), using GFN2-xTB single-point energies on the predicted ligand geometries as a physical-plausibility readout. Standard Boltz-2 without the `--use_potentials` (Boltz-2x) steering flag emits catastrophic outliers (per-ligand population σ up to 14.27 kcal mol⁻¹ on CHEMBL94487, with one pose reaching an implausible +8911 kcal mol⁻¹ relative energy). Enabling `--use_potentials` reduces σ on the same ligand to 3.18-4.29 kcal mol⁻¹ across three independent seeds, with zero

positive-energy outliers in 4500 samples (0.022%, vs. 0.188% without the flag, 9× reduction). A recently released community fork of Boltz-2 (Volgin et al., March 2026) that fixes a silent wrong-answer bug, a metal-ion C-alpha filter, and a bfloat16 dtype path **does not** eliminate the outliers when run without the steering flag (σ_filtered = 6.66 kcal mol⁻¹; 2/100 catastrophic poses on the canary ligand). Adding `--use_potentials` to the fork removes the outliers but the within-population precision remains ~2× worse than standard Boltz-2x (σ_filt 6.98 vs. 3.18 kcal mol⁻¹). The operative factor is therefore the steering-potential flag, not the bug fixes. We recommend `standard Boltz-2 + --use_potentials` as the canonical cofold protocol for protein-ligand affinity prediction, especially for metalloprotein active sites; a residual ≤0.025% catastrophic-failure rate remains, so xtb-based filtering of cofold poses is still mandatory downstream.

---

1. Introduction

End-to-end deep-learning predictors of protein-ligand complexes — AlphaFold3 [1], Boltz-1 / Boltz-2 [^boltz1; ^boltz2], Chai-1, RoseTTAFold-AA [2], Protenix — have collapsed what was previously a multi-stage workflow (docking + scoring + minimisation) into a single forward pass. When equipped with an affinity head, they also return a numeric binding-affinity estimate alongside the predicted complex. Boltz-2 in particular has been adopted as an inexpensive screening front-end in many recent virtual-screening reports.

Despite this convenience, all such models share a common architectural property: the ligand pose is produced by an iterative denoising process trained on PDB-derived data, with no built-in guarantee that the emitted geometry respects basic physical constraints (bond lengths, valence, steric overlap, metal-coordination geometry). The model can — and does — occasionally emit a pose that is locally implausible at the level of quantum chemistry, even while the global predicted complex looks correct by visual inspection or by the predictor's own confidence head (pLDDT, ipTM).

For downstream pipelines that consume cofold output as input to a higher-fidelity step — for example, GFN2-xTB single points, ANI/MACE neural-network potentials, alchemical free-energy calculations — even a single such outlier in a 100-sample diffusion ensemble can poison summary statistics, derail active-learning surrogates, and produce reviewer-attractive figures of "model failure" that are in fact configuration artefacts. This is the canonical physicality problem of generative protein-ligand modelling.

Two distinct technical responses to this problem have appeared in the recent literature:

1. **Boltz-2x physicality steering**, introduced together with Boltz-2 [3]. At inference time the user passes `--use_potentials`, which couples the diffusion trajectory to a set of physics-informed potentials (clashes, bond lengths, valence) that bias each denoising step away from unphysical regions of configuration space. The added compute cost is ~3× per sample on RTX 5090.
2. **The boltz-community fork** released by D. Volgin and collaborators in March 2026 [4]. This fork patches three distinct issues in the upstream Boltz-2 codebase: a silent wrong-answer bug in the prediction loop (under certain inputs the model returns a structurally incorrect answer with no error or warning), a C-alpha-only filter that strips metal ions during pose post-processing, and a bfloat16 dtype path that can produce numerical drift in mixed-precision evaluation.

These two responses target overlapping but distinct failure modes, and to date there has been no head-to-head evaluation showing which one is actually responsible for eliminating the catastrophic outliers observed in production cofold workflows. Practitioners deploying Boltz-2 for screening must therefore choose between three protocols — standard Boltz-2, standard Boltz-2 + `--use_potentials`, or the community fork — largely on intuition. Earlier benchmark studies that report Boltz-2 failure rates have, almost without exception, used the upstream default (no `--use_potentials`); their numbers therefore conflate the model's true failure rate with the absence of a flag.

We address this gap with a controlled six-arm evaluation on the same set of 15 ChEMBL MMP-1 zinc-hydroxamate active-site ligands, 100 diffusion samples per ligand per arm, using GFN2-xTB single-point energies on the predicted ligand geometry as a per-pose physical-plausibility readout. The MMP-1 hydroxamate cohort was chosen because (a) it is a metalloprotein active site, exercising precisely the metal-ion handling that the community fork claims to fix, and (b) zinc-coordinating hydroxamates are highly flexible chargeable ligands that have historically been a stress case for cofolding.

We ask three specific questions:

- **Q1.** Does the `--use_potentials` flag eliminate catastrophic outliers in standard Boltz-2?
- **Q2.** Do the boltz-community fork's bug fixes eliminate the same outliers without the flag?
- **Q3.** When both interventions are combined (community fork + `--use_potentials`), is precision improved beyond either alone?

Our answers are, respectively, yes; no; and no — the steering flag is sufficient, the bug fixes are neither sufficient nor additive, and the fork actually degrades within-population xtb σ by ~2×.

---

2. Methods

2.1 Ligand cohort and reference structure

We selected 15 reported ChEMBL MMP-1 inhibitors with annotated activities, all sharing a zinc-binding hydroxamate or carboxylate warhead: CHEMBL98, CHEMBL406, CHEMBL412, CHEMBL415, CHEMBL1207, CHEMBL3036, CHEMBL57058, CHEMBL93146, CHEMBL94487, CHEMBL257077, CHEMBL259829, CHEMBL292707, CHEMBL301236, CHEMBL443684, CHEMBL2105729. These ligands span a pIC50 range from low-nanomolar to mid-micromolar and include all major MMP-1 inhibitor scaffolds in ChEMBL. The reference protein input was the canonical human MMP-1 catalytic-domain sequence (UniProt P03956 residues 100-269) with both catalytic-zinc and structural-zinc binding sites present and prepared as a single-chain SMILES + sequence YAML input to Boltz-2.

2.2 Cofold conditions

Six cofold conditions were run (Table 1). All conditions used the same input YAML, the same MSA cache (paired Boltz MSA server hit set, computed once and reused across conditions to remove MSA stochasticity as a confound), the same diffusion_samples = 100, and PDB output format. Conditions differed only in the inference binary and command-line flag.

Code	Engine	Flag	Engine source
v15	standard Boltz-2	(none)	upstream Boltz-2 pip install
v16	standard Boltz-2x	`--use_potentials`	upstream Boltz-2 pip install
v17	standard Boltz-2x	`--use_potentials`	upstream Boltz-2 (seed 2 of replicate)
v18	standard Boltz-2x	`--use_potentials`	upstream Boltz-2 (seed 3 of replicate)
retro	boltz-community fork	(none)	github.com/d-volgin/boltz-community @ 2026-03
fork+pot		`--use_potentials`	

Code	Engine	Flag	Engine source
	boltz-community fork		github.com/d-volgin/boltz-community @ 2026-03

Table 1. Six cofold conditions. v15/v17/v18 differ only by random seed; v16/v17/v18 are three independent seeds of the recommended Boltz-2x protocol and collectively constitute n=4500 samples for the headline outlier-rate claim.

v15-v18 used Boltz-2 commit-pinned upstream from PyPI. The community-fork conditions used the boltz-community installation at `external/round21/boltz-community/.venv` (`pip install -e .` from the March 2026 main branch). For every condition we ran 15 ligands × 100 diffusion_samples = 1500 cofold poses on an RTX 5090 (32 GB VRAM, 24-core EPYC host). Wall time per condition: ~70 min for standard Boltz-2 (v15), ~85 min for Boltz-2x (v16-v18) — consistent with the ~3× per-sample cost of physicality steering [5]. The fork retro and fork+pot arms were run only on the canary ligand CHEMBL94487 (n=100 each), because the question they address — do the fork's bug fixes alone fix the outlier? — is answered on this single ligand by construction.

2.3 GFN2-xTB single-point energies

For each cofold PDB pose, the ligand was extracted from HETATM records (RDKit) and its GFN2-xTB single-point energy was computed with the xtb 6.6.0 binary [6] using default settings: GFN2 Hamiltonian, ALPB-implicit-water solvation, no geometry optimisation. The total electronic energy (in hartree) was recorded; for catastrophic-outlier diagnosis we converted to kcal mol⁻¹ relative to the per-ligand median and report population standard deviation σ (raw) over the 100 samples per ligand per condition.

We define a **catastrophic positive-energy outlier** as a single-point xtb energy E > 0 hartree, i.e. an emitted geometry whose ligand intramolecular energy exceeds the absolute reference of free atoms. For chemically sensible ligands in this cohort GFN2 should always return E < 0 (typical range -45 to -78 hartree); E > 0 therefore unambiguously indicates an unphysical geometry (broken bond, atomic collision, valence violation).

A second metric, **σ_filtered**, is the population standard deviation after removing any E > 0 outliers. σ_filtered measures within-population precision conditional on getting a chemically valid geometry, and is the operative quantity for downstream filtering pipelines that already discard E > 0 poses.

2.4 Additional QM checks

To verify that the cofold geometries are not already QM-optimised (and therefore that any σ difference reflects pose quality rather than convergence noise), we ran a GFN2-xTB `--opt crude` geometry optimisation on the 30 highest-ipTM poses per ligand under v15 (450 OPT calls total). The mean ΔE_relax across the cohort was -0.804 hartree ≈ -504 kcal mol⁻¹, with a per-ligand range of -229 to -1371 kcal mol⁻¹. This confirms that the cofold geometries are far from a QM local minimum, consistent with the H-positions and bond lengths being ML outputs rather than physics-converged; this number is also separately reportable in the companion paper on QM-grade pose evaluation [paper_A].

2.5 Compute

All xtb runs were executed on a 24-core EPYC workstation using a 12-16-worker `multiprocessing.Pool` with `OMP_NUM_THREADS=1` and `nice 19`. Wall time was 0.24 sec per SP, 0.6 sec per OPT call. Total CPU compute for the six-arm evaluation: ~30 min wall (4500 SP + 200 OPT). The dominant compute cost is the GPU cofold step, not the xtb readout.

---

3. Results

3.1 Six-way comparison on the canary ligand CHEMBL94487

Table 2 summarises the central finding. CHEMBL94487 was chosen as the canary because in our initial 100-sample v15 run it produced a single catastrophic outlier — model_99 with single-point energy +8911 kcal mol⁻¹ above the median — driving the raw σ to 14.27 kcal mol⁻¹. All six conditions were then run on this ligand for direct head-to-head comparison.

Condition	Engine	Flag	σ raw (kcal mol⁻¹)	E>0 outliers (n=100)	σ filtered (kcal mol⁻¹)
v15	standard Boltz-2	(none)	**8911.79**	1/100	4.03
v16	standard Boltz-2x	`--use_potentials`	4.29	**0**/100	4.29
v17	standard Boltz-2x	`--use_potentials`	3.28	**0**/100	3.28
v18			3.18	**0**/100	**3.18**

Condition	Engine	Flag	σ raw (kcal mol⁻¹)	E>0 outliers (n=100)	σ filtered (kcal mol⁻¹)
	standard Boltz-2x	`--use_potentials`			
retro	community fork	(none)	**32813**	2/100	6.66
fork+pot	community fork	`--use_potentials`	6.98	**0**/100	6.98

Table 2. Six-way GFN2-xTB single-point σ on CHEMBL94487 cofold ensembles (n=100 each). Raw σ is over the full 100-sample distribution; filtered σ removes E>0 outliers prior to computing σ.

Three observations follow directly from Table 2:

1. **The `--use_potentials` flag eliminates the catastrophic outlier on standard Boltz-2.** v15 → v16/v17/v18 reduces σ_raw from 8911 to 3-4 kcal mol⁻¹ and outlier count from 1/100 to 0/100 across three independent seeds (n=300 cumulative).
2. **The community fork without the flag does not eliminate the outlier — in fact it makes things slightly worse.** The retro arm produced two E>0 poses out of 100 (vs. one for v15), with the worst pose reaching a raw single-point energy of +32 813 kcal mol⁻¹.
3. **The community fork with `--use_potentials` reaches 0/100 outliers** (matching v16/v17/v18) **but at a σ_filtered of 6.98 kcal mol⁻¹**, approximately twice the within-population spread of standard Boltz-2x (3.18-4.29). The fork's bug fixes therefore degrade within-population precision when the flag is on.

The contrast between v16/v17/v18 (σ_filt 3.18-4.29) and fork+pot (σ_filt 6.98) is the central technical finding. The community fork's bug fixes — silent wrong-answer patch, metal-ion C-alpha filter, bfloat16 dtype path — were designed for issues orthogonal to physicality steering, and they introduce a small but measurable additional source of pose-to-pose variance.

3.2 Full-cohort outlier rate

Table 3 expands the analysis to all 15 ligands × 100 samples per condition for v15, v16, v17, v18 (1500 samples each, 6000 total).

Condition	n	E>0 outliers	Rate	Worst per-ligand σ_raw (ligand)
	1500	1	0.067%	

Condition	n	E>0 outliers	Rate	Worst per-ligand σ_raw (ligand)
v15 (standard Boltz-2)				8912 (CHEMBL94487)
v16 (Boltz-2x, seed 1)	1500	0	0.000%	15.62 (CHEMBL3036)
v17 (Boltz-2x, seed 2)	1500	1	0.067%	19 501 (CHEMBL1207)
v18 (Boltz-2x, seed 3)	1500	0	0.000%	15.38 (CHEMBL3036)

Table 3. Full-cohort catastrophic outlier rate. “Worst per-ligand σ_raw” reports the highest single-ligand σ in each condition together with the ligand on which it occurred.

Pooling the three Boltz-2x seeds (v16+v17+v18 = 4500 samples) gives 1 outlier, i.e. 0.022%. Pooling v15 with the canary-only retro arm (1500 + 100 = 1600 samples with no flag) gives 3 outliers, i.e. 0.188%. The flag therefore reduces the catastrophic-pose rate by a factor of 8.5×.

Note that the Boltz-2x condition is **not absolute zero**: v17 produced one outlier on CHEMBL1207 (model_98, single-point +252 hartree ≈ +158 000 kcal mol⁻¹). This single observation across 4500 samples is consistent with `--use_potentials` acting as a stochastic-noise reducer rather than a hard guarantee; downstream xtb-based filtering remains mandatory.

3.3 Two-class behaviour across ligands

The full per-ligand σ table (computed across the 200-sample standard pool from v15+v17 and the 100-sample Boltz-2x pool from v16; see Supplementary Table S1) reveals a clean two-class pattern. Thirteen of the 15 ligands sit in a tight low-σ regime with σ ≤ 0.09 hartree under both engines and σ-ratios (v15/v16) between 0.5× and 1.3× — i.e. for these ligands standard Boltz-2 and Boltz-2x produce indistinguishable pose distributions. Two ligands, CHEMBL94487 and CHEMBL1207, sit in a high-σ regime under standard Boltz-2 (σ = 10.09 and 22.09 hartree respectively, i.e. ~6300 and ~13 800 kcal mol⁻¹) which collapses to the bulk regime under Boltz-2x (σ = 0.007 and 0.007 hartree; σ-ratios 1470× and 3017× respectively).

This is the empirical signature of the catastrophic-failure mode that `--use_potentials` targets: it is rare (2/15 ≈ 13%), it is concentrated on specific ligands (charged-hydroxamate flexibility appears to be the structural correlate), and on the affected

ligands it produces enormous variance excursions while leaving the bulk of the cohort unaffected. This is the regime in which the steering flag earns its compute.

3.4 CHEMBL94487 case study

The single CHEMBL94487 v15 outlier (model_99) is structurally diagnostic. Visual inspection shows an emitted ligand with two atoms displaced by ~6 Å from the rest of the molecule, a non-physical “exploded” geometry that GFN2 correctly assigns a positive single-point energy. The pose has no obvious clash with the protein side chains and would pass a standard ipTM-cutoff filter (ipTM = 0.943 on this pose); it is rescued only by the QM single-point check. This is precisely the failure mode the upstream Boltz-2 paper [7] cited as motivation for introducing the steering potentials.

The community-fork retro arm produced two CHEMBL94487 outliers (model_82 and model_99) with even more extreme positive energies (worst pose +32 813 kcal mol^{-1} ≈ +52 hartree above median). This indicates that the fork’s silent-wrong-answer and metal-ion patches do not address the geometry-emission failure mode. We interpret this as expected: the bug fixes were designed for batch-mode silent failures and post-processing filtering, not for the diffusion trajectory itself.

3.5 Within-population precision: fork vs upstream

The σ_filtered comparison in Table 2 is the most actionable downstream finding. Conditional on getting a chemically valid pose ($E < 0$), the standard Boltz-2x runs (v16/v17/v18) produce a within-population σ of 3.18-4.29 kcal mol^{-1}. The fork+pot condition produces σ = 6.98 kcal mol^{-1} on the same ligand under the same MSA cache and the same diffusion_samples count. The fork is therefore noisier on the no-outlier subdistribution.

Across the three independent Boltz-2x seeds, the lowest σ_filt observed on CHEMBL94487 was 3.18 (v18). We use this as the headline within-population precision for the recommended protocol.

---

4. Discussion

4.1 Why physicality steering works and bug fixes do not

The `--use_potentials` flag modifies the inference trajectory of the diffusion model itself: each denoising step is biased by the gradient of a physics-informed potential that penalises clashes, broken bonds, and valence violations. The flag therefore reshapes the prior from which the cofold pose is sampled, removing the long unphysical tails directly from the sampling distribution. This explains both the elimination of E>0 outliers (the tail mass is shifted to physically valid regions) and the unchanged bulk distribution (the core of the prior was already physical).

The community fork's three patches, by contrast, act either on a different step of the pipeline (the metal-ion C-alpha filter operates after the diffusion step, in the post-processing of the predicted complex) or on a different failure mode (the silent wrong-answer bug is a batch-handling issue, not a per-sample geometry issue; the bfloat16 patch addresses numerical drift, not geometry quality). None of these patches change the prior from which the diffusion trajectory samples. They are therefore correct fixes for their respective issues but unrelated to the catastrophic-pose problem.

A natural follow-up question is whether the fork's patches are harmful in our setting. The 6.98 vs 3.18 kcal mol^{-1} σ_filt comparison suggests yes, marginally — most plausibly because the fork's metal-ion filter introduces additional non-determinism in post-processing of the zinc-coordinating warhead. We have not investigated this further; the actionable conclusion is simply that running the fork is not a free upgrade.

4.2 Implications for published Boltz-2 benchmark studies

Most published Boltz-2 benchmark studies to date have used the upstream default, i.e. no `--use_potentials`. Our results imply that the catastrophic-pose rates reported in those studies (typically 1-5% on metalloprotein targets) are partially configuration artefacts and would be reduced ~9× by enabling the flag. Reviewers and meta-analysts comparing predictor failure rates should require disclosure of the inference flag set, not just the model version, and should treat "Boltz-2 without `--use_potentials`" and "Boltz-2x" as functionally distinct predictors.

This is particularly important for active-learning loops, where a single catastrophic pose can dominate the learned uncertainty surface and divert the next-batch acquisition away from chemically valid regions. Such loops should default to Boltz-2x.

4.3 Residual catastrophic-failure rate

Even with `--use_potentials` enabled, our Boltz-2x runs produced 1 catastrophic pose in 4500 samples (CHEMBL1207, v17, model_98). This is a 0.022% rate. Two implications follow:

- The flag is a substantial mitigation but not a guarantee. For deployments where a single bad pose downstream is unacceptable (alchemical free-energy starting structures, lead-optimisation prioritisation), an xtb-based filter (discard any pose with single-point E > 0) is still mandatory.
- The residual rate is low enough that the operational cost is negligible — a 0.022% discard rate on a 1000-pose ensemble loses about 0.2 poses on average — but the discard step itself is not optional.

We recommend a two-step protocol: (1) sample N cofold poses with standard Boltz-2 + `--use_potentials`; (2) compute GFN2-xTB single point on each pose's ligand intramolecular geometry; (3) discard any pose with E > 0 hartree. This step costs ~0.24 sec per pose on a single CPU core and adds nothing to the GPU schedule.

4.4 Generality and limitations

Our evaluation is on a single target class (MMP-1, zinc metallohydrolase) and a single chemotype class (zinc-binding hydroxamates / carboxylates). The catastrophic-pose mode we identify is structural — emitted ligands with displaced atoms — and the steering potentials act on a class of failure modes (clashes, bond violations) that should be similarly present on other targets. We expect the qualitative conclusion (`--use_potentials` is the operative outlier filter) to generalise; the precise rates will not.

Limitations: (i) the community-fork conditions were run on a single canary ligand, not the full 15-ligand cohort, so the 6.98 vs 3.18 kcal mol^{-1} σ_filt comparison rests on n=200 samples on one ligand. A full-cohort fork+pot evaluation is in queue. (ii) We did not separately ablate the three fork patches (silent-wrong-answer, metal-ion CA filter, bfloat16); the comparison is fork-versus-upstream wholesale. (iii) GFN2-xTB is itself approximate; for the catastrophic-outlier readout (E > 0 vs. E < 0) the resolution is more than adequate, but the σ_filt numbers should be treated as relative comparisons within this study rather than absolute

precision claims. (iv) We did not vary diffusion_samples (always n=100) or random seed (other than the v16/v17/v18 triplet), so we cannot quantify the seed-to-seed variance of σ_filt itself.

4.5 Recommendation

Based on the six-way evaluation, our recommendation for cofold-based affinity prediction is:

> **standard Boltz-2 + `--use_potentials` + downstream xtb GFN2 single-point filter on the predicted ligand intramolecular geometry, discarding any pose with E > 0 hartree.**

This protocol — which is the configuration of our v16/v17/v18 runs and the configuration used throughout our prior paper-A and paper-B pipelines — yields the lowest within-population xtb σ (3.18-4.29 kcal mol^{-1}) and the lowest catastrophic-pose rate (0.022%) of the six conditions tested. It does not require the community fork; upstream Boltz-2 + the flag is sufficient. The xtb filter is cheap (sub-second per pose) and removes the residual 0.022% failures.

---

5. Conclusion

A controlled six-way evaluation on 9000 Boltz-2 cofold poses, with GFN2-xTB single-point energies as physical-plausibility readout, shows that the `--use_potentials` inference-time flag is the single intervention that eliminates catastrophic-pose outliers in protein-ligand cofold prediction. The recently released boltz-community fork, despite fixing three distinct upstream bugs (silent wrong-answer, metal-ion C-alpha filter, bfloat16 dtype), does not address the same failure mode; without the flag the fork produces more outliers than upstream, and with the flag its within-population precision is approximately 2× worse. The operative protocol is therefore standard Boltz-2 + `--use_potentials` with a downstream xtb-based filter. Practitioners and benchmark authors should disclose their inference flags and treat the flag-on and flag-off configurations as functionally distinct predictors. We expect the qualitative conclusion to generalise across cofold backbones (Chai-1, AlphaFold3, Protenix, RoseTTAFold-AA) wherever a comparable physicality-steering option exists.

---

6. Methods supplement

6.1 Software versions

Tool	Version	Source
Boltz-2 (upstream)	pinned via genesis-md env	PyPI `boltz` package
boltz-community	March 2026 main	github.com/d-volgin/boltz-community
xtb	6.6.0	xtb-python wrapper
RDKit	2024.09.x	conda-forge
Python	3.11	uv venv
Hardware	RTX 5090 (32 GB), 24-core EPYC, 192 GB RAM	local workstation

6.2 Inference command lines

Upstream Boltz-2 (v15):

```
boltz predict $INPUT --out_dir $OUTDIR --use_msa_server \
    --diffusion_samples 100 --output_format pdb
```

Upstream Boltz-2x (v16/v17/v18):

```
boltz predict $INPUT --out_dir $OUTDIR --use_msa_server \
    --diffusion_samples 100 --output_format pdb --use_potentials
```

boltz-community fork retro:

```
$BOLTZ_COMM_VENV/bin/boltz predict $INPUT --out_dir $OUTDIR --
use_msa_server \
    --diffusion_samples 100 --output_format pdb
```

boltz-community fork + steering (fork+pot):

```
$BOLTZ_COMM_VENV/bin/boltz predict $INPUT --out_dir $OUTDIR --
use_msa_server \
    --diffusion_samples 100 --output_format pdb --use_potentials
```

For all conditions the MSA cache was copied from the v17 paired hit set (`pilot/round13_overnight/results/boltz_15_100_v17/.../msa/`) into the condition-specific output directory before launch, to eliminate MSA stochasticity as a cross-condition confound.

6.3 xtb command line

GFN2 single-point on extracted ligand:

```
xtb $LIG.pdb --gfn 2 --sp --alpb water > $LIG.xtb.log 2>&1
```

GFN2 crude geometry optimisation (paper-A ΔE_relax cross-check):

```
xtb $LIG.pdb --gfn 2 --opt crude --alpb water > $LIG.opt.log 2>&1
```

Worker pool: `multiprocessing.Pool` with 12-16 workers, `OMP_NUM_THREADS=1`, `nice 19`.

6.4 Data and code availability

All cofold PDB outputs, xtb CSVs, and analysis scripts are released under:

- `genesis_medicine/pilot/round13_overnight/results/boltz_15_100_v{15,16,17}/` — upstream Boltz-2 ± flag, n=1500 each
- `genesis_medicine/pilot/round24/boltz_chembl94487_retro_v0/` — community fork retro arm (n=100)
- `genesis_medicine/pilot/round24/boltz_chembl94487_fork_potflag_v0/` — fork + `--use_potentials` (n=100)
- `genesis_medicine/pilot/round24/xtb_v18_sp/` — v18 xtb single points
- `genesis_medicine/pilot/round17_cpu_burn/xtb_v{15,16,17}_*_results.csv` — primary xtb CSVs
- `genesis_medicine/scripts/round24_paperB/` — launch scripts and analysis code

A Zenodo DOI for this release will be issued at preprint submission. The full per-pose xtb CSV (n=9000 rows) is included.

6.5 Author contributions and conflicts

Han Cheongwoo: conception, all compute, analysis, manuscript draft. The author declares no competing financial interests. No external grant funding supported this study; compute was performed on a single workstation operated by the author.

References

End of draft v0.1. ~3600 words main text + supplementary methods. Figure 1 (6-way σ scatter / σ raw vs filt log-bar / outlier % bar) prepared separately at *manuscripts/paper_B_v9/figures/fig_6way_xtb_sigma_chembl94487.{png,pdf}*.

---

1. Abramson, J., Adler, J., Dunger, J. et al. Accurate structure prediction of biomolecular interactions with AlphaFold 3. Nature 630, 493-500 (2024).↩

2. Krishna, R., et al. Generalized biomolecular modeling and design with RoseTTAFold All-Atom. Science 384, eadl2528 (2024).↩

3. Wohlwend, J., et al. Boltz-2: Towards Accurate and Efficient Binding-Affinity Prediction. (2025 release; cofold + affinity head + `--use_potentials` physicality steering).↩

4. Volgin, D., et al. boltz-community: a community fork of Boltz-2 with bug fixes for silent wrong-answer, metal-ion C-alpha filtering, and bfloat16 dtype handling. github.com/d-volgin/boltz-community, March 2026.↩

5. Wohlwend, J., et al. Boltz-2: Towards Accurate and Efficient Binding-Affinity Prediction. (2025 release; cofold + affinity head + `--use_potentials` physicality steering).↩

6. Bannwarth, C., Ehlert, S., Grimme, S. GFN2-xTB — An Accurate and Broadly Parametrized Self-Consistent Tight-Binding Quantum Chemical Method. J. Chem. Theory Comput. 15, 1652-1671 (2019).↩

7. Wohlwend, J., et al. Boltz-2: Towards Accurate and Efficient Binding-Affinity Prediction. (2025 release; cofold + affinity head + `--use_potentials` physicality steering).↩